

Persuas — 5-Minute Live Demo Script

Razorpay AI Buildathon · Track 03 — AI Revenue Recovery | Live app: persuasnav.vercel.app

Segment	Time	Purpose
1. Hook	0:00 – 0:20	Attack the "number went up" claim
2. The holdout proof	0:20 – 0:55	Show the only real measurement
3. The decision queue	0:55 – 1:25	Show it mostly refuses to act
4. DO_NOT_ACT, run live	1:25 – 2:10	Prove refusal is reasoned, not passive
5. The twin test	2:10 – 3:30	The centrepiece — what only a processor can do
6. The honest report card	3:30 – 4:05	Show it grading its own failures
7. Real Razorpay, live	4:05 – 4:35	Prove it's deployed and connected
8. Close / the Razorpay case	4:35 – 5:00	Why this belongs inside Razorpay

Rehearse this once at full pace with a timer before presenting — 5 minutes goes faster live than it reads on paper. If you're cut short, drop segment 6 first and compress segment 3 into one sentence; never cut segments 2 or 5. Segment 5 is the one nobody else in the room can show.

1 · Hook

0:00 – 0:20

ON SCREEN: Landing page loading, or a blank screen before you share it

"Every recovery product you'll see today is going to show you a graph where a number goes up after their AI does something. That number is usually wrong — not on purpose, but because most failed payments recover on their own, with no help. If you can't measure against a control group, you can't tell your intervention from a coincidence."

"So before I show you what my agent does, I'll show you the one thing most of these systems can't show you: proof that it caused anything at all."

2 · The holdout proof

0:20 – 0:55

ON SCREEN: Landing page hero animation running, then the live stats strip below it

"This is the actual experiment my agent runs, live, right now. Two lanes of failed payments. Left lane — the agent acts. Right lane — the holdout — gets nothing, on purpose. Watch the right bar. It fills too, because failed payments recover on their own. That's the part every other recovery product quietly takes credit for."

"The gap between these two bars is the only number on this whole page that's a measurement instead of an assertion — a real randomized holdout, run automatically on every cohort this system touches."

"On this run, that gap is a [read live lift]% lift, and it means [read live phantom-share]% of what a normal dashboard would call 'recovered revenue' would have happened anyway."

3 · The decision queue

0:55 – 1:25

ON SCREEN: Click "Open the console" → Batch run screen, full table visible

"Here's the working queue — every decision the agent made on its own, with no human triggering it. Look at the chips: ACT, WAIT, EXPERIMENT, DO NOT ACT, BLOCKED. Four of those five outcomes are the agent choosing not to act."

"It only acts when the *lower bound* of its estimated value is still positive — the worst-case-and-it-still-wins number, not the hopeful one. That single rule is why it doesn't behave like every other agent that intervenes on everything it sees."

4 · DO_NOT_ACT, run live

This is the segment most demos skip because refusal is less visually exciting than action — which is exactly why it's the strongest proof point in the room. Run it live, don't just describe it.

1:25 – 1:50

ON SCREEN: Switch to a terminal window, already sized large enough to read on screen-share; have it pre-positioned before you start talking

"Let me actually run the refusal case live, not just show you a screenshot of it."

Type and run: `npm run batch -- quiet`

"This scenario has nothing wrong in it at all — no incident was injected. Watch what it does."

1:50 – 2:05

ON SCREEN: Terminal output scrolling — narrate over it as it prints

"It scans the cohorts, checks for a change point, and finds nothing that clears its significance threshold — corrected for the fact that it just ran hundreds of these tests in one window, so ordinary noise doesn't get mistaken for a real incident."

"It concludes DO_NOT_ACT and stops. No discount sent, no message fired, no cost spent — because spending anything against noise would be negative expected value the moment you account for what it costs to act."

"This is the behavior that's actually hard to build. Anyone can make an agent that acts. Making one that knows when **not** to, and can prove why, is the harder engineering problem — and it's the one this track is actually about."

2:05 – 2:10

ON SCREEN: Back to the console, Batch run screen, filter dropdown set to DO_NOT_ACT

"And here's that same discipline at scale — every DO_NOT_ACT call across the full evaluation, one filter click away, each with its own reasoning attached."

5 · The twin test — the centrepiece

This is the segment no other submission can run, because it needs data from merchants they do not have. Give it the time. If you are running long, cut anything else first.

2:10 – 2:35

ON SCREEN: Console → Rail network. Stay on the top strip and the first callout.

"Now the part I actually want to be judged on. Every system you have seen today, including mine up to this slide, has the same blind spot — and it is not a modelling blind spot, it is a physics one."

"A merchant cannot tell 'this issuer is degraded' from 'my checkout broke for that issuer's customers.' Same rail. Same hours. Same cohort. Same failure code. Identical data. There is no model that separates those, because the information that would is not in one merchant's data at all."

2:35 – 3:05

ON SCREEN: Scroll to the two World panels. Point at World B — the red one — as you say the line about being wrong.

"So here is the same incident in two worlds. In World A the issuer really is down and the whole fleet is hit. In World B one merchant broke its own retry timing. Same seed, same rail, same severity, same duration, same failure reason."

"Alone, my engine calls World A right — and calls World B issuer degradation, which is wrong. It blames the bank for its own bug."

"With anonymous signals from the other merchants on that rail: one of thirty-nine is degraded. Not the rail — you. And the same engine, unchanged, now says retry timing."

3:05 – 3:30

ON SCREEN: Scroll to the top stat strip again, then to the 'what crosses the boundary' panel for the last line.

"Measured across twenty-four seeded pairs: fifty percent alone, eighty-three with the network. And that fifty is not bad luck — split it and the engine is a hundred percent right when the issuer really is down and zero percent right when the fault is its own. Sixteen cases corrected, none broken."

"And what crosses the merchant boundary is a z-statistic, an effect size, a sample size and an onset hour. No identity, no customers, no amounts, no raw events — and nothing is reported at all below five independent contributors."

6 · The honest report card

3:30 – 4:05

ON SCREEN: Click into Evaluation screen, scroll to the baseline comparison table, then the recall/confusion matrix section

"This is the part most teams here won't have: a report card the agent didn't get to grade itself. 140 blind scenarios it's never seen, tested against five baselines, including 'discount everyone' and 'retry everything.' Every baseline either overclaims or loses money once you subtract the holdout."

"And here's the part that took discipline to leave in: it misses about a third of real incidents. This shows exactly which ones, and why — it's not hiding its own error rate to look better on stage."

7 · Real Razorpay, live

4:05 – 4:35

ON SCREEN: Show the provenance badge on any console screen, then switch tabs to the Razorpay dashboard showing the paid test payment link

"This isn't a static mockup. It's deployed live, and this badge confirms a real Razorpay test-mode connection. I ran an actual payment through it end to end — a real payment link, a real test payment, a signature-verified webhook hitting my deployed endpoint."

"It also refuses to run at all against a live key — this system creates payment links, and it has no business holding real credentials during a hackathon build."

8 · Close — the Razorpay case

4:35 – 5:00

ON SCREEN: Back to the landing page, or hold on the final console screen

"The one honest gap in my own evaluation is that issuer-level failures are genuinely hard to see from a single merchant's data — that pattern really resolves at cross-merchant scale. Razorpay sees every merchant. What I've built is a working, self-testing proof of this decision logic — detect, diagnose, decide, measure, comply — at merchant scale. That's the argument for why it becomes more valuable inside Razorpay than beside it."

"Thank you — happy to open any decision on that queue and walk through exactly why it did or didn't act."

Appendix — likely judge questions

Keep this page nearby but don't read from it live — know these cold.

"Isn't this just detect-and-discount with extra steps?"

"The detect-and-discount version is the one thing every other team here can build in a weekend. The holdout, the matched-control causal test, and the self-reported failure rate are the parts that take actual statistical design up front — you can't retrofit a holdout after you've already shipped 'act on everything.'"

"How do I know your eval numbers aren't favorable to your own method?"

"Baselines get the same detector output I do — I didn't handicap them. The corpus also includes 30 null windows with nothing wrong at all, specifically to check for false positives, and it reports zero."

"What's not real here?"

"Transaction history and incidents are simulated and labeled as such on every screen — a provenance badge never lets you forget which is which. Razorpay orders, payment links, and webhook verification are real test-mode API calls."

"How deep is the Razorpay integration, really?"

"Honestly, today it's ingestion-depth: real test-mode orders, payment links, and a signature-verified webhook — not yet platform-level routing control. That's exactly the next milestone, and it's a natural one because the engine already treats live events and simulated ones through the identical contract."

"Couldn't a merchant game the network, or reverse-engineer a competitor?"

"Two gates. Nothing is published below five independent contributors, and that check runs before any statistic is computed, so a thin rail never produces a number at all. And the verdict is a count of how many merchants independently moved, not a pooled average — so one large contributor cannot drag it. There is a test for exactly that adversarial case. What it is not is differential privacy: there is no noise budget, and repeated queries across windows still leak slowly. That's in the eval doc."

"What breaks this at real scale?"

"The JSON-file ledger is a prototype store — it needs to become a real database before production. That's documented, not hidden, in my runbook."